LeenaElDebaawy / DIY

Code Issues Pull requests Agents Actions Projects

Security and quality Insights Settings

main DIY / README.md

LeenaElDebaawy Prepare repository handover documentation and setup files 2076c8f · 7 minutes ago

702 lines (508 loc) · 32.4 KB

Preview Code Blame Raw

Knitra — AI-Assisted Home Repair Diagnosis

Knitra is a locally hosted home repair assistant for UK homeowners. A user uploads a photo or video of a household fault, optionally describes it in writing, and receives a structured assessment: a classification of the problem, a routing decision based on risk level and evidence quality, and — where appropriate — a step-by-step repair plan with tools, materials, and safety guidance.

All inference runs locally via Ollama or HuggingFace Transformers. No images, audio, or repair data leave the machine.

The system is built around a single principle: **a repair assistant is only useful if it is honest about what it does not know.** Generating a confident-sounding plan for a situation it cannot safely assess is treated as a failure mode, not a feature. The pipeline spends as much effort deciding *whether* to advise as it does producing advice.

Quick Start

Prerequisites: Python 3.11+, Git, Docker Desktop, Ollama

```
# 1. Pull the required AI models into Ollama (one-time, ~15 GB total)
ollama pull qwen2.5vl:7b          # Stage 1 — image classification
ollama pull mistral:7b-instruct   # Stage 2 — tools, materials, root causes
ollama pull llama3.1:8b           # Stage 3 — repair steps and chat

# 2. Start PostgreSQL
```

```
docker-compose up -d

# 3. Backend  (new terminal, from repo root)
cd backend
cp .env.example .env              # edit DATABASE_URL and SECRET_KEY
pip install -r requirements.txt
alembic upgrade head
python -m uvicorn main:app --host 0.0.0.0 --port 8000

# 4. Frontend  (another terminal, from repo root)
cd frontend
pip install -r requirements.txt
python main.py
```

Open **http://localhost:3000** in your browser.

> Video analysis and voice input require additional packages — see `backend/requirements-video.txt` and set `VIDEO_ANALYSIS_ENABLED=true` in `.env`. Admin Quality Dashboard and judge features require `JUDGE_ENABLED=true` and `ollama pull qwen2.5:7b`.

Full setup details, environment variable reference, and a step-by-step QA checklist are in Section 7 — Setup and Installation.

Table of Contents

1. Project Overview
2. Key Features
3. Architecture
4. AI Pipeline
5. Output Modes / Routing
6. Admin Quality Dashboard
7. Setup and Installation
8. Environment Variables
9. Running the App
10. Testing and QA Checklist
11. API Overview
12. Safety and Limitations
13. Handover Notes

1. Project Overview

The problem

Someone facing a household fault usually has three unsatisfying options: pay a professional for something they could have handled themselves, attempt the repair without proper guidance, or sift through online advice that may not apply to their situation.

A naïve AI assistant creates a fourth option that is arguably worse than all three: it produces a confident-sounding plan regardless of whether the evidence supports one. For a domain where mistakes can mean flooding, electrocution, or structural damage, confident guesswork is a liability.

What Knitra does

- Accepts an image or short video of a household repair problem
- Runs a structured visual diagnosis using a local multimodal model
- Routes each case to one of six output modes based on risk level, evidence quality, and problem type — not a single confidence threshold
- For low-risk, clearly visible problems: generates a specific repair plan with tools, materials, ordered steps, and safety notes
- For high-risk or unclear situations: escalates or asks targeted questions rather than producing advice the evidence does not support
- Tracks user feedback and — for admins — runs an automated pipeline quality audit via a local judge model

2. Key Features

Analysis and diagnosis

Feature	Description
Image case analysis	Photos are passed to a local vision model for structured classification into problem type, trade, severity, risk level, and DIY suitability.
Video case analysis	Short clips are processed by extracting frames, which feed into the same Stage 1 classification. Requires `VIDEO_ANALYSIS_ENABLED=true`.
Six output modes	Each case routes deterministically to one of six outcomes. The same diagnosis always produces the same route.

Feature	Description
Trade-specific output	Stage 2 and Stage 3 prompts are specialised by trade (carpentry, plumbing, electrical, plastering, etc.), not generic.
One-way safety escalation	Post-processing rules can raise (never lower) the model's own risk signals before routing.

User interaction

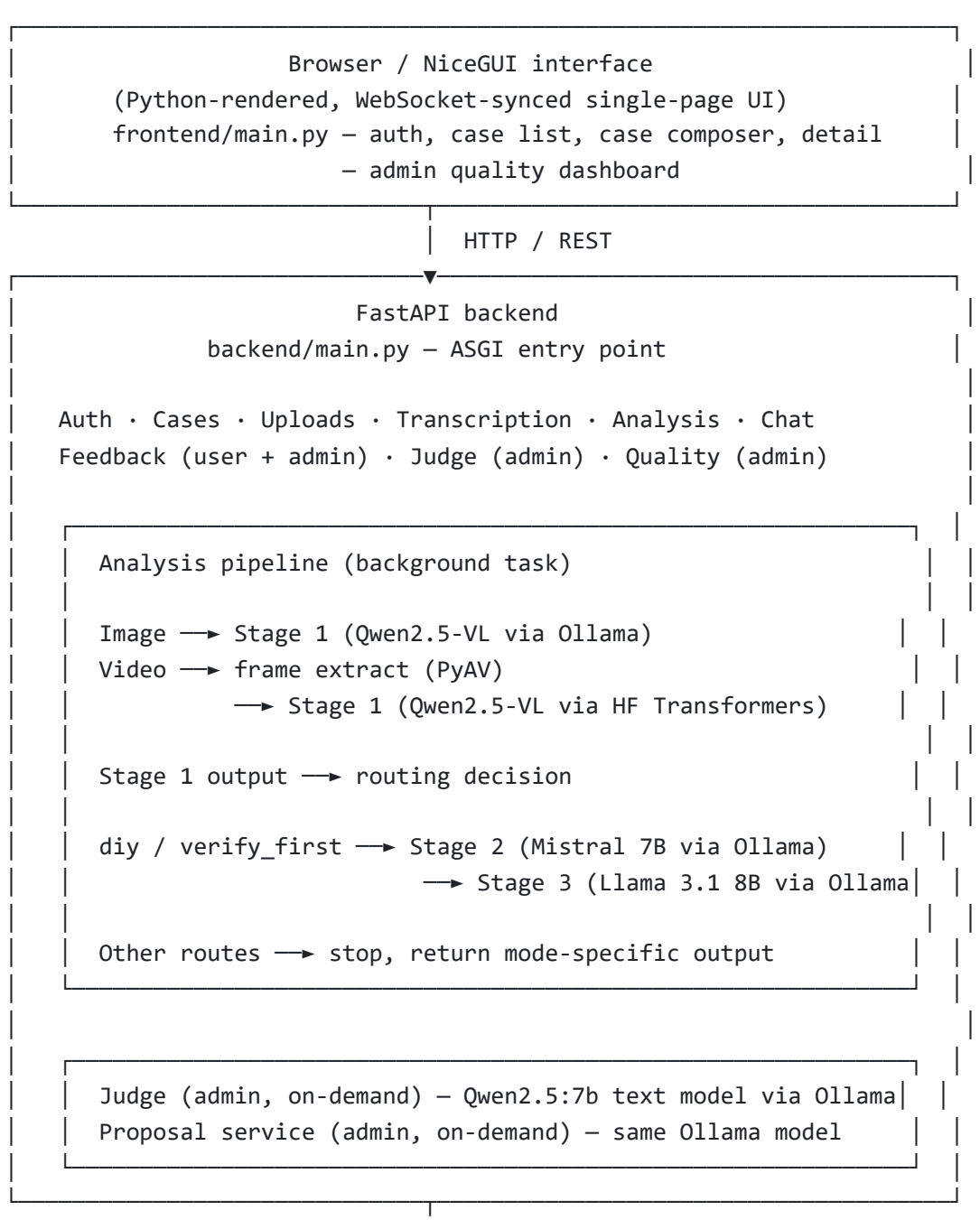
Feature	Description
Ask Knitra chat	Per-case contextual chat assistant anchored to the specific diagnosis and repair output. Opens after analysis completes.
Voice input	Dictated messages are transcribed locally using faster-whisper and placed in the message input.
Follow-up photos in chat	Users can attach additional photos mid-conversation; the vision model describes them and the assistant incorporates that context.
Clarification with memory	When more information is needed, the system asks targeted questions and carries all prior answers forward across rounds.
Hazard acknowledgement gate	High-risk cases require the user to read and confirm a safety notice before the conversational assistant opens.
User feedback	Users can rate each analysis (Helpful / Partially / Not helpful) and select reason chips from a structured list.
Feedback reason chips	Chips cover: wrong diagnosis, too cautious, not cautious enough, not detailed enough, wrong tools/materials, steps unclear, wrong professional recommendation, advice didn't match upload, repeated a clarification question, advice felt unsafe, other.

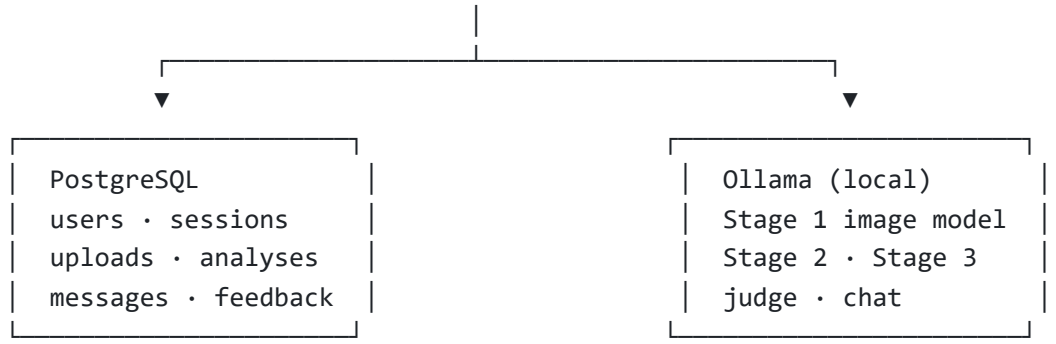
Admin features

Feature	Description
Judge evaluation	Admin-only endpoint runs a local text model to audit completed analyses for safety, routing, grounding, hallucination risk, and clarity.

Feature	Description
Admin Quality Dashboard	Dedicated `/admin/quality` page combining judge results and user feedback into a comprehensive QA view.
Advisory fix proposals	AI-generated (Ollama-backed), admin-only proposals for addressing detected quality patterns — advisory only, never applied automatically.
Nearby services	Placeholder / coming-soon feature for surfacing professional services relevant to the case location.

3. Architecture





Key components

Component	File	Purpose
Frontend	`frontend/main.py`	NiceGUI single-page interface including admin quality dashboard
Backend entry	`backend/main.py`	FastAPI ASGI entry point — start with `uvicorn main:app`
Analysis pipeline	`backend/app/services/analysis_service.py`	Stages 1–3, routing, safety post-processing, clarification
Video analysis	`backend/app/services/video_analysis_service.py`	Frame extraction (PyAV) + HF Transformers inference path
Judge service	`backend/app/services/judge_service.py`	Pipeline quality auditor (admin-only, Ollama)
Feedback service	`backend/app/services/feedback_service.py`	User feedback creation and admin aggregation
Quality service	`backend/app/services/quality_service.py`	Joins judge + feedback data for

Component	File	Purpose
		the dashboard
Proposal service	`backend/app/services/proposal_service.py`	AI-generated advisory fix proposals (admin-only, Ollama)
Database	PostgreSQL via Docker	Users, sessions, uploads, analyses, messages, feedback
Migrations	`backend/alembic/versions/`	Schema history managed by Alembic

4. AI Pipeline

Stage 1 — Visual classification

One or more images (or extracted video frames) are passed to the Stage 1 model alongside any user-provided text description. Stage 1 returns:

- Problem type and trade category
- Severity, confidence, risk level, DIY suitability
- Lists of what is and is not visible in the media
- Clarifying questions, if critical information is absent

Post-processing rules may raise (never lower) the model's risk signals for specific high-risk categories (electrical, gas, structural, water heater, hidden mould/damp).

Stage 2 — Repair planning

Runs only when the case routes to `diy` or `verify_first`. Takes Stage 1 output as context and produces:

- Essential and optional tools, materials, protective gear
- Estimated time, ranked likely causes with confidence percentages
- A repair confidence assessment citing the specific Stage 1 evidence

Trade-specific profiles are injected into the Stage 2 prompt (e.g. carpentry profile prohibits Allen keys for concealed hinges, requires Pozi/Phillips screwdrivers).

Stage 3 — Step generation

Takes Stage 1 and Stage 2 outputs and produces the final user-facing output: numbered steps in WHAT / WHY / RESULT format, difficulty level, safety notes, and guidance on when to call a professional.

Judge model — Pipeline quality auditor

The judge is a separate, admin-only evaluation step that runs *after* Stage 3 completes. It is **not** a second diagnosis — it receives only the pipeline's own text outputs (Stage 1 observations, Stage 2 routing, Stage 3 final output) and checks the pipeline's internal consistency.

The judge evaluates:

- **Safety**: are hazard claims grounded in Stage 1 observations?
- **Routing**: was the output mode appropriate given the evidence?
- **Grounding**: does Stage 3 stay within what Stage 1 actually observed?
- **Hallucination risk**: are there claims not supported by the input?
- **Warning completeness**: are relevant safety warnings present?
- **Clarity**: is the language appropriate for a UK homeowner?

The judge produces a structured verdict (`pass` , `needs_review` , `unsafe`), a score, dimension scores, issue tags, and an actionable findings list. Results are stored per analysis and surfaced in the Quality Dashboard.

Video-specific Stage 1 fields

When a video case is processed, Stage 1 additionally returns:

Field	Description
`key_moments`	Frame descriptions of diagnostically important moments
`what_can_be_confirmed`	What the video frames allow the model to verify
`what_cannot_be_confirmed`	What cannot be determined from available frames
`hidden_damage_risk`	Whether unseen damage is likely (`low` / `medium` / `high`)
`temporal_confidence`	How consistently the problem appears across frames (0–100)
`motion_detected`	Whether movement relevant to the fault is visible

5. Output Modes / Routing

The routing decision runs on the adjusted Stage 1 output. Gates are checked in priority order:

Priority	Condition	Output mode
1 (highest)	`scope != 'repair'`	`out_of_scope`
2	`professional_assessment_recommended=True` or `risk_level` is `high` / `critical`	`assessment_first`
3	`needs_clarification=True` or confidence below floor	`needs_clarification`
4	High hidden damage risk, observed hazard, or conditional DIY suitability	`verify_first`
5	Low-risk maintenance task	`care`
6 (default)	Everything else	`diy`

Gate 2 takes priority over Gate 3. A high-risk case bypasses the clarification gate entirely — a clarification question does not change a safety recommendation.

Mode	User receives
`diy`	Full repair plan: tools, materials, ordered steps, safety notes
`verify_first`	Full repair plan plus a specific pre-check to perform first
`assessment_first`	What to do immediately, what to avoid, which professional to contact. No repair steps.
`needs_clarification`	Summary of knowns and unknowns, targeted questions
`out_of_scope`	Explanation of why the system cannot assist
`care`	Material-specific care or maintenance guidance

6. Admin Quality Dashboard

The Quality Dashboard is available at `/admin/quality`. It is only accessible to users whose email is listed in `ADMIN_EMAILS` in `backend/.env`.

The dashboard joins judge evaluation results and user feedback into a single QA view across six tabs.

Overview tab

- **QA Health Score** — composite metric combining user helpful rate (40%), average judge score (40%), and confirmed issue rate (20%). Displayed as a colour-coded banner (green / amber / red).
- **Judge–User Agreement %** — percentage of paired cases where judge and user reached the same conclusion (both good or both flagged).
- **Top 3 issues this week** — most common judge tags and user feedback chips in the last 7 days.
- **Summary counts** — total cases, judged cases, feedback submissions, and alignment distribution.

User Feedback tab

- Per-case feedback with rating, reason chips, and free-text comment
- Filter by rating, reason chip, and output mode
- Reset filters button
- Active filter count

Judge Evaluation tab

- Per-case judge results with verdict, score, issue tags, and mode
- Filter by verdict, mode, and score band
- Reset filters button

Alignment tab

- **4 signal cards**: Both Good (judge passed + user said Helpful), Both Flagged (judge flagged + user complaint), Safety Conflict (judge flagged Unsafe + user said Helpful), Calibration Gap (judge passed + user unhappy)
- Each card shows count, percentage of paired cases, and up to 3 example case links
- **Cases Needing Attention** table — cases with confirmed issues or safety conflicts, with priority highlighting and quick-open links

Recurring Issues tab

- Aggregated issue tags from judge evaluations
- Shows frequency and source for each pattern

Suggestions tab

- System-generated improvement suggestions derived from recurring patterns
- Each suggestion shows the pipeline area affected, trigger issues, occurrence count, and signal source (judge / user feedback / both)
- **Generate fix proposal** button — calls local Ollama to produce a structured advisory proposal including: title, problem observed, evidence summary, specific suggested change, pipeline area, expected impact, risks and cautions
- Proposals are advisory only — never applied automatically

7. Setup and Installation

Prerequisites

- Python 3.12 or newer
- Docker Desktop — for PostgreSQL
- Ollama — installed and running locally
- A CUDA-capable GPU is recommended for video analysis; CPU-only is supported but slow
- Node.js is **not** required

Step 1 — Start the database

```
docker compose up -d db
```

This starts PostgreSQL on port 5433 using the credentials in `docker-compose.yml`.

Step 2 — Pull Ollama models

One-time download; all inference is local after this.

```
# Core pipeline models
ollama pull qwen2.5vl:7b          # Stage 1 — image classification
ollama pull mistral:7b-instruct   # Stage 2 — tools, materials, root causes
ollama pull llama3.1:8b           # Stage 3 — repair steps and chat

# Judge and proposal model (admin QA features)
ollama pull qwen2.5:7b            # Judge evaluation and fix proposals
```

Verify Ollama is running:

```
curl http://localhost:11434/api/tags
```

Step 3 — Configure the backend

```
cd backend
cp .env.example .env
# Edit .env — see §8 for all required variables
```

Key variables to set: `DATABASE_URL`, `ADMIN_EMAILS`, `JUDGE_ENABLED`, `JUDGE_MODEL`.

Step 4 — Backend virtual environment and migrations

```
cd backend

# Create and activate virtual environment
python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt

# Run all database migrations
alembic upgrade head
```

Step 5 — Frontend virtual environment

In a separate terminal:

```
cd frontend

python -m venv .venv

# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate

pip install -r requirements.txt
```

8. Environment Variables

All variables are read from `backend/.env`. The example file is at `backend/.env.example`. **Never commit `.env`.**

Database

Variable	Description
`DATABASE_URL`	PostgreSQL connection string (asyncpg format)
`DATABASE_ECHO`	Set `true` to log all SQL statements to stdout

Auth and admin access

Variable	Description
`SECRET_KEY`	Secret used for token signing
`ADMIN_EMAILS`	Comma-separated list of email addresses that have admin access to judge endpoints and the Quality Dashboard

Ollama / image pipeline

Variable	Description
`OLLAMA_BASE_URL`	Ollama endpoint, e.g. `http://localhost:11434`
`STAGE_1_MODEL`	Vision model for image Stage 1 classification (e.g. `qwen2.5vl:7b`)
`STAGE_2_MODEL`	Text model for Stage 2 planning (e.g. `mistral:7b-instruct`)
`STAGE_3_MODEL`	Text model for Stage 3 generation and chat (e.g. `llama3.1:8b`)
`PIPELINE_RETRY_COUNT`	Number of retries per stage on invalid model output
`OLLAMA_KEEP_ALIVE`	How long Ollama keeps a model loaded after last use
`STAGE_1_NUM_CTX`	Stage 1 context window (must fit image tokens + prompt)

Judge and Quality Dashboard

Variable	Description
`JUDGE_ENABLED`	Set `true` to enable the judge model and advisory proposals. Set `false` in production if Ollama is not available.
`JUDGE_MODEL`	Ollama model used for judge evaluation and fix proposals (e.g. `qwen2.5:7b`)

Video pipeline

Variable	Description
`VIDEO_ANALYSIS_ENABLED`	Set `true` to enable video case analysis
`VIDEO_INFERENCE_MODE`	`local` (HF Transformers) or `cloud` (remote vLLM endpoint)
`VIDEO_MODEL_LOCAL`	HuggingFace model ID for local video inference
`VIDEO_NFRAMES`	Number of frames sampled per video (default: 12)
`VIDEO_MAX_NEW_TOKENS`	Token budget for the video model response
`VIDEO_USE_4BIT`	Set `true` to enable 4-bit quantisation (reduces VRAM usage)

Frontend

Variable	Description
`BACKEND_URL`	URL the frontend uses to reach the backend API (default: `http://127.0.0.1:8000`)

9. Running the App

Backend

```
cd backend
```

Windows (recommended):

```
.\.venv\Scripts\python.exe -m uvicorn main:app --host 0.0.0.0 --port 8000
```

macOS / Linux:

```
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

> **Do not use `--reload` if you plan to run video analysis.** The file-watcher restarts the process mid-inference, which kills in-progress video analysis and can cause model loading to duplicate. `--reload` is safe for image-only development.

Frontend

In a second terminal:

```
cd frontend
```

Windows:

```
.\.venv\Scripts\python.exe main.py
```

macOS / Linux:

```
python main.py
```

Service URLs

Service	URL
Frontend	http://localhost:3000
Backend	http://localhost:8000
API docs	http://localhost:8000/docs
Ollama	http://localhost:11434
PostgreSQL	localhost:5433

10. Testing and QA Checklist

These are recommended manual checks, not automated tests. Run them after any change to the pipeline, routing, or prompts.

Core user flow

- [ ] Register a new user account
- [ ] Create an image case (upload a photo of a household fault)
- [ ] Trigger analysis and wait for completion
- [ ] Confirm the output mode is appropriate for the image
- [ ] Open Ask Knitra chat and send a follow-up question
- [ ] Submit feedback (rating + reason chips + optional comment)

Case types

- [ ] **DIY case** — clear, low-risk fault; expect `diy` or `verify_first`; Stage 2/3 should run
- [ ] **Emergency case** — electrical fault or gas leak image; expect `assessment_first`; no repair steps
- [ ] **Out-of-scope case** — decorative/non-repair task; expect `out_of_scope`
- [ ] **Clarification case** — ambiguous image; expect `needs_clarification`; answer the questions; confirm re-analysis runs

Video analysis (if `VIDEO_ANALYSIS_ENABLED=true`)

- [ ] Upload a short video clip of a repair fault
- [ ] Trigger analysis; confirm video-specific Stage 1 fields appear in the result
- [ ] Confirm the same Stage 2/3 path runs as for images

Admin Quality Dashboard

- [ ] Log in with an admin email (`ADMIN_EMAILS` in `.env`)
- [ ] Navigate to `/admin/quality`
- [ ] Confirm normal user accounts cannot access `/admin/quality` (expect 403)
- [ ] Run the judge on a completed case (`POST /api/v1/cases/{id}/judge`)
- [ ] Reload the dashboard — confirm the judged case appears in Judge tab
- [ ] Test verdict filter and "↺ Reset" button in the Judge tab
- [ ] Test rating filter and reset in the User Feedback tab
- [ ] Test alignment filter and reset in the Alignment tab
- [ ] Check the Alignment tab 4-card view shows correct counts
- [ ] Open the Suggestions tab — if suggestions appear, click "Generate fix proposal →"
- [ ] Confirm the proposal dialog shows a loading state, then AI-generated content (or a static fallback if `JUDGE_ENABLED=false`)

Safety checks

- [ ] Confirm hazard acknowledgement gate blocks chat until notice is confirmed on `assessment_first` cases
- [ ] Confirm judge model cannot be run without admin credentials (expect 403)

11. API Overview

All routes under `/api/v1/` require a bearer token except registration, login, and the health probe. Admin routes additionally require the requesting user to be in `ADMIN_EMAILS`.

Authentication

Method	Path
POST	/api/v1/auth/register
POST	/api/v1/auth/login
GET	/api/v1/auth/me
POST	/api/v1/auth/logout

Cases and uploads

Method	Path
POST / GET	/api/v1/cases
GET / PATCH / DELETE	/api/v1/cases/{id}
POST / GET	/api/v1/cases/{id}/uploads

Analysis and chat

Method	Path
POST	/api/v1/cases/{id}/analyse
GET	/api/v1/cases/{id}/analysis
GET / POST	/api/v1/cases/{id}/chat
POST	/api/v1/cases/{id}/chat/stream

Feedback

Method	Path	Access
POST	/api/v1/cases/{id}/feedback	Any authenticated user
GET	/api/v1/admin/feedback/overview	Admin only
GET	/api/v1/admin/feedback/priority	Admin only
GET	/api/v1/admin/feedback/modes	Admin only
GET	/api/v1/admin/feedback/comments	Admin only
GET	/api/v1/admin/feedback/safety	Admin only
PATCH	/api/v1/admin/feedback/{id}	Admin only

Judge and Quality Dashboard

Method	Path	Access
POST	/api/v1/cases/{id}/judge	Admin only
GET	/api/v1/cases/{id}/judge	Admin only
GET	/api/v1/admin/judge/summary	Admin only
GET	/api/v1/admin/quality/summary	Admin only
POST	/api/v1/admin/quality/fix-proposal	Admin only

Other

Method	Path
POST	/api/v1/transcribe
GET	/health

12. Safety and Limitations

AI advice is not a substitute for a qualified professional

Knitra provides guidance based on visual analysis by a locally running language model. It is intended to help users make an informed decision about whether they can handle a repair themselves — not to replace a qualified tradesperson's assessment.

For any case involving electrical wiring, gas supply, load-bearing structures, or active flooding, the system routes to `assessment_first` with an explicit recommendation to contact a professional. Users should follow that recommendation.

Emergency cases

Cases flagged as active emergencies (gas leak, electrical hazard, flooding) display immediate action guidance and direct the user away from DIY repair steps. The hazard acknowledgement gate ensures the safety notice is seen before the conversational assistant opens.

Judge model scope

The judge audits the pipeline's own text outputs for internal consistency. It does not re-examine the original image or video. If Stage 1 missed something in the visual evidence, the judge cannot catch it. The judge is a pipeline quality auditor, not a second visual diagnosis.

Advisory proposals

Fix proposals generated via the Quality Dashboard are advisory only. They are AI-generated text suggesting changes a human developer should review. No proposal is applied automatically, and no files are modified by the proposal service.

Local model variability

Output quality is bounded by the size of the local models. Mistral 7B (Stage 2) and Llama 3.1 8B (Stage 3) may produce inconsistent tool lists or step orderings across runs. The judge and clarification mechanisms surface this rather than hiding it, but cannot fully compensate for model limitations.

Video analysis speed

The first video analysis in a process takes approximately 30 seconds to load the model into memory, plus inference time. On CPU-only machines, video analysis is not practical for routine use.

13. Handover Notes

QA state

The admin Quality Dashboard at `/admin/quality` is the recommended starting point for reviewing pipeline quality before handover. Run the judge on a representative sample of recent cases, check the Alignment tab for Safety Conflicts, and review the Suggestions tab for any recurring patterns.

Feedback and judge records from earlier development sessions are preserved in the database and will appear in the dashboard.

Alembic migrations

The migration chain in `backend/alembic/versions/` covers the full schema. A fresh database is brought up to date with:

```
alembic upgrade head
```

Migrations in the chain (in order):

- `dd9275c4f79f` — full initial schema
- `c17eeef6f32c` — plan server default
- `b4e7f2a9c0d1` — hazard acknowledgement on sessions
- `e5f6a7b8c9d0` — video fields on uploads
- `f1a2b3c4d5e6` — feedback table
- `a1b2c3d4e5f6` — reason chips on feedback

Files that must not be committed

- `backend/.env` — contains database credentials, `SECRET_KEY`, and `ADMIN_EMAILS`
- Generated log files (`*.log`) at the backend and frontend root
- `backend/uploads/` — user-uploaded media
- `.nicegui/` — NiceGUI build cache (already in `.gitignore`)

JUDGE_ENABLED flag

`JUDGE_ENABLED=true` is required for judge evaluation and advisory fix proposals to work. If Ollama is not running or the judge model is not pulled, set `JUDGE_ENABLED=false` — the dashboard will still show user feedback and static suggestion summaries, and the proposal button will display a clear disabled-state message rather than erroring.

Known gaps and future improvements

- **Selective visual spot-check**: a future judge mode that re-examines the original image for specific claims flagged by the text-based judge
- **Case review status**: a field to mark dashboard cases as reviewed/resolved
- **QA report export**: export the dashboard summary to PDF or CSV for handover documentation
- **Nearby services integration**: currently a placeholder; a future version could surface relevant local tradespeople based on the case's trade category and location
- **Frontend display of `likely_causes` and `repair_confidence_reasoning`** : these fields are produced by Stage 2 but are not yet displayed in all frontend views
- **Automated test suite**: the `backend/spikes/` directory contains development scripts for replaying Stage 1 outputs and testing routing logic, but there is no formal test suite

Technology summary

Component	Technology
Backend framework	FastAPI (async) with Python 3.12
ORM	SQLAlchemy 2.0 (async)

Component	Technology
Database	PostgreSQL (via Docker)
Migrations	Alembic
Frontend	NiceGUI
Schema validation	Pydantic v2
Image Stage 1 model	Qwen2.5-VL 7B via Ollama
Video Stage 1 model	Qwen2.5-VL 7B via HuggingFace Transformers
Stage 2 model	Mistral 7B Instruct via Ollama
Stage 3 + chat model	Llama 3.1 8B via Ollama
Judge + proposals model	Qwen2.5 7B (text) via Ollama
Video frame extraction	PyAV (`av` package)
Speech-to-text	faster-whisper (CPU, local)
Image processing	Pillow